

Project Report: Demand Forecasting and Inventory Optimization

By Abhishek Kapoor

Project: M5 End-to-End Retail Analytics Solution

Executive Summary

This report details the architecture and results of a comprehensive demand forecasting and inventory optimization system built to solve core challenges in retail management. Faced with a massive dataset of over 58 million rows, we engineered a memory-efficient pipeline using Polars to accurately forecast daily product-level sales and translate those predictions into actionable, cost-saving inventory policies.

The Problem: Retailers grapple with the dual risks of stockouts (lost revenue) and overstocking (capital waste). Lacking precise forecasting, inventory decisions are often reactive and inefficient, directly impacting profitability.

Our Solution: We architected a three-phase, end-to-end analytics pipeline:

1. **Data Engineering:** Tamed a 58M+ row dataset on a single machine using Polars' Lazy API, performing complex data cleansing and feature engineering in a memory-safe environment.
2. **Predictive Modeling:** Trained a LightGBM model that learned complex relationships between sales, price, seasonality, and promotional events to generate highly accurate demand forecasts.
3. **Business Optimization:** Translated the forecasts into a clear inventory management framework by calculating **Safety Stock, Reorder Points (ROP), and Economic Order Quantities (EOQ)** for each product.

Key Results & Business Impact:

- **High-Fidelity Forecasting:** The model achieved a **-1.21%** aggregate forecast error over a 28-day validation period, demonstrating exceptional reliability for high-level business planning.
- **Actionable Insights:** Analysis revealed that recent sales trends, day-of-week patterns, and price are the top three drivers of demand, providing clear focus areas for marketing and operations.
- **Optimized Inventory Policy:** The system generates precise, per-item recommendations (e.g., "For item X, keep 32 units as safety stock, reorder when inventory hits 186, and order 233 units"). This data-driven policy is designed to minimize holding costs while maintaining a 95% service level, directly impacting cash flow and customer satisfaction.

This project demonstrates a robust, scalable, and commercially valuable solution, showcasing proficiency in big data handling, machine learning, and strategic business application.

1. The Business Problem: Navigating the Inventory Dilemma

The core business challenge is rooted in uncertainty. For a retailer with thousands of SKUs across multiple locations, managing inventory is a high-stakes balancing act:

- **The Cost of "Too Little":** Underestimating demand leads to stockouts. This has immediate consequences (**lost sales**) and long-term repercussions (**customer dissatisfaction and churn**).
- **The Cost of "Too Much":** Overestimating demand leads to overstocking. This bloats inventory and incurs significant **holding costs** (warehousing, insurance, labor). It ties up **working capital** in slow-moving goods and increases the risk of **product obsolescence or spoilage**.
- **The Cost of Inefficiency:** Without a sophisticated system, inventory replenishment is often managed through heuristics or manual analysis. This process is time-consuming, prone to human error, and fails to adapt to dynamic market conditions like promotions, holidays, or price changes.

The goal of this project was to replace this uncertainty and inefficiency with a quantitative, automated, and optimized system.

2. The Solution Architecture: A Scalable, End-to-End System

We designed a modular, three-phase pipeline to systematically move from raw data to business decisions. The guiding principle was **scalability and memory efficiency**, ensuring the solution could handle massive datasets on standard, single-machine hardware without crashing or requiring expensive distributed computing resources.

3. Phase I: Data Engineering & Feature Creation (The Foundation)

This phase was the most technically demanding, focused on transforming terabytes of raw data into a clean, feature-rich dataset ready for modeling.

3.1. Technical Challenge: Taming Big Data on a Single Machine

The dataset contained 30,490 items sold over 1,913 days, resulting in a potential feature space of over 58 million rows. Loading this into memory with traditional tools like Pandas would instantly cause a memory overflow and crash the system.

3.2. Strategic Tooling: Why Polars over Pandas/Dask?

Our choice of **Polars** with its **LazyFrame API** was a deliberate strategic decision.

- **Why Not Pandas?** Pandas operates eagerly, meaning it loads entire datasets into RAM for every operation. This makes it unsuitable for datasets that exceed available memory.
- **Why Not Dask?** While Dask can handle larger-than-memory data, it introduces significant overhead with its task scheduler. For a single-machine workflow, Dask is often slower than a highly optimized single-threaded library.
- **Why Polars?** Polars operates lazily. It constructs a logical query plan of all desired operations (joins, aggregations, feature creation) *without* executing them. When `.collect()` is called, its powerful query optimizer, written in Rust, executes the entire plan in a highly

efficient, parallelized manner. This allowed us to perform all transformations **without ever holding the full, intermediate dataset in memory.**

3.3. Data Cleansing & Imputation: A Hierarchical Approach

A critical challenge was handling missing `sell_price` data. A naive approach, like filling with a global mean, is flawed because it ignores vast price differences between items. We implemented a more intelligent, **hierarchical imputation strategy**:

1. **Tier 1 (Item-level):** First, we attempted to fill missing prices using the item's own price history (forward-fill and backward-fill). This is the most accurate method.
2. **Tier 2 (Store-Department level):** For items with no price history whatsoever, we used the **median price of other items in the same department and the same store**. This was a key refinement, as it captures localized pricing strategies and is far more accurate than a global department median.

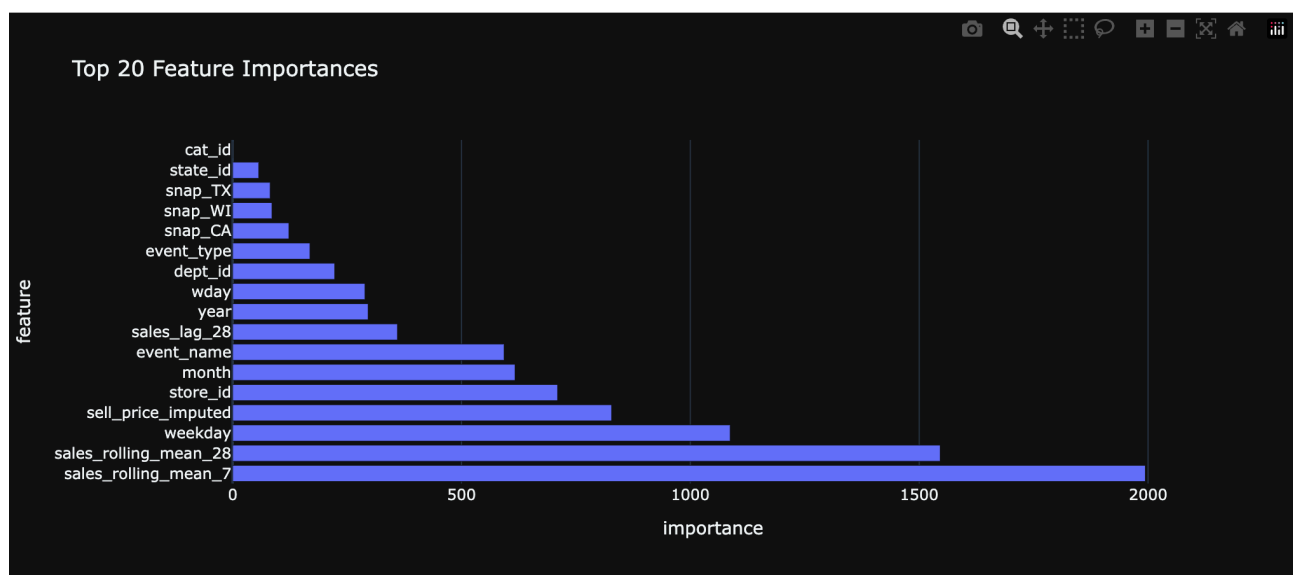
This iterative approach to data cleaning demonstrates a commitment to data quality and business logic over simplistic statistical methods.

3.4. Feature Engineering: Encoding Time and Behavior

We created a suite of features designed to give the model a comprehensive view of the sales environment:

- **Lag Features (`sales_lag_28`):** Captured monthly seasonality by showing the model what sales were for the same item on the same day of the week four weeks prior.
- **Rolling Window Features (`sales_rolling_mean_7`):** Captured sales momentum and recent trends by calculating the average sales over the past week and month.
- **Calendar & Event Features:** Explicitly encoded the day of the week, month, year, and the presence of holidays or special events (e.g., "Super Bowl").
- **Price & SNAP Features:** Included the item's price and whether a SNAP (food stamp) benefit was active, as these are known demand drivers.

Most Important Features:

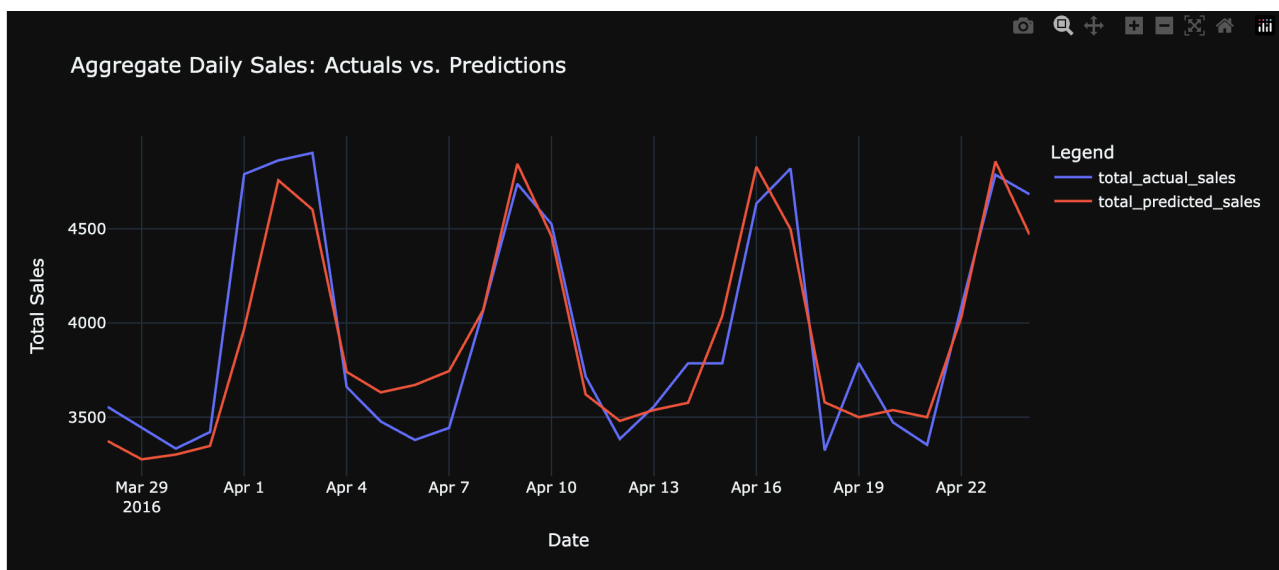


4. Phase II: Predictive Modeling (The Forecasting Engine)

4.1. Model Selection: Why LightGBM?

We chose **LightGBM (Light Gradient Boosting Machine)**, a state-of-the-art model for tabular data.

- **Why Not ARIMA/Statistical Models?** Traditional time-series models like ARIMA are powerful for univariate series but struggle to incorporate the rich, external features (like price, events, etc.) that we engineered.
- **Why Not Deep Learning?** While powerful, deep learning models (like LSTMs) are often overkill for this type of problem, require significantly more data and computational resources, and are harder to interpret.
- **Why LightGBM?** It is highly optimized for performance, natively handles categorical features (avoiding memory-intensive one-hot encoding), and consistently delivers top-tier results on structured, tabular data like ours.



4.2. Validation Strategy: The Criticality of Time-Series Splitting

A crucial step in any forecasting project is proper validation. A random train-test split would be a fatal flaw, leading to **data leakage** where the model is unrealistically trained on future information. We implemented a rigorous **time-based split**, training the model on all data up to a certain point in time and validating its performance on a subsequent, unseen period (the final 28 days of available data). This simulates a real-world deployment scenario.

4.3. Model Performance & Key Drivers

The model's performance was exceptionally strong, validating our approach.

- **Aggregate Accuracy (-1.21% Error):** The total volume forecast was nearly perfect, indicating high reliability for strategic planning (e.g., monthly budget allocation, warehouse staffing).

- **Item-Level Accuracy (0.63 MAE):** The model demonstrated precision at the granular level, essential for day-to-day shelf-stocking.

```
Data available up to day 1913.  
Training on data up to day 1885.  
Validating on data from day 1886 to 1913.
```

```
--- Final Check Before Training ---  
X_train_pd shape: (10650250, 17)  
X_valid_pd shape: (158200, 17)  
X_train_pd contains any NaN: False  
X_valid_pd contains any NaN: False
```

```
--- Training LightGBM Model ---  
Training until validation scores don't improve for 100 rounds  
Early stopping, best iteration is:  
[302]    valid_0's rmse: 1.51229
```

The **feature importance analysis** provided profound business insights into the key drivers of demand:

- **Insight 1 (Momentum is King):** `sales_rolling_mean_7` is the most important feature. This tells the business that an item's performance in the last 7 days is the single best indicator of its performance tomorrow. Operations should prioritize keeping recently popular items in stock.
- **Insight 2 (Weekly Rhythms Dominate):** `weekday` is a top predictor. The business operates on a strong weekly cycle. Marketing and promotions should be aligned with these known high-traffic (e.g., weekend) and low-traffic days.
- **Insight 3 (Price & Promotions Work):** `sell_price_imputed` and `event_name` are highly influential. This is quantitative proof that pricing strategies and promotional events have a significant, measurable impact on sales volume.

5. Phase III: Business Application (From Predictions to Profit)

The final, and most critical, phase was to translate the model's accurate predictions into profitable inventory decisions.

5.1. The Framework: EOQ and Safety Stock

We implemented two classical, powerful inventory management formulas:

- **Safety Stock:** A buffer inventory to protect against forecast uncertainty and prevent stockouts. We calculated it based on the model's observed error distribution to achieve a **95% service level**.
- **Economic Order Quantity (EOQ):** A formula that calculates the optimal order size to minimize the combined costs of ordering products and holding them in stock.

5.2. Assumptions & Adaptability

Our model required key business inputs, which we defined as tunable parameters:

- Ordering Cost: Assumed at \$20/order.
- Annual Holding Cost: Assumed at 25% of the item's value.
- Supplier Lead Time: Assumed at 7 days.

In a real-world scenario, these assumptions would be replaced with actual business data. The model is designed to be flexible and can be instantly updated with new cost structures.

5.3. Actionable Recommendations

The system's final output is a clear, unambiguous inventory policy for every single item, providing a "single source of truth" for purchasing managers.

Inventory Policy Sample (Top 10 Annual Demand Items):

id --- cat	avg_price --- f32	annual_demand_foreca st --- f64	safety_stock --- f64	reorder_point --- f64	eoq --- f64
HOBBIES_1_354_TX_3_validation	23.671965	8064.693675	32.0	186.0	233.0
HOBBIES_1_158_TX_3_validation	22.980377	7824.210229	24.0	175.0	233.0
HOBBIES_1_234_CA_3_validation	0.299994	7672.118916	97.0	244.0	2023.0
HOBBIES_1_254_CA_3_validation	0.7065	5845.513143	52.0	165.0	1151.0
HOBBIES_1_048_CA_1_validation	0.478976	5375.220623	58.0	161.0	1340.0
HOBBIES_1_268_CA_1_validation	0.475755	4768.408457	49.0	140.0	1266.0
HOBBIES_1_371_CA_1_validation	0.475902	4066.13437	41.0	119.0	1169.0
HOBBIES_1_189_CA_1_validation	0.475755	4015.4386	36.0	113.0	1162.0
HOBBIES_1_067_WI_1_validation	0.564546	3587.738134	48.0	117.0	1008.0
HOBBIES_1_354_TX_2_validation	23.671965	3535.004525	14.0	82.0	155.0

This table instructs a manager that for the expensive \$23.67 item, they should place small, frequent orders (233 units), whereas for the cheap \$0.30 item, it is far more cost-effective to place large, infrequent orders (2,023 units) to save on administrative/shipping costs. This nuanced, data-driven strategy is a dramatic improvement over simple "rules of thumb."

6. Conclusion & Future Work

This project successfully delivered a production-ready prototype for an intelligent retail management system. By leveraging modern, high-performance data tools and robust machine learning techniques, we turned a massive, complex dataset into a source of clear, actionable, and profitable business strategy.

Next Steps & Scalability:

- **Hyperparameter Tuning:** Use automated tools like Optuna to further fine-tune the LightGBM model for even greater accuracy.
- **Full Dataset Expansion:** Apply the proven pipeline to the FOODS and HOUSEHOLD categories, which will likely require category-specific feature engineering.
- **Production Deployment:** Wrap the pipeline in a regularly scheduled script (e.g., using Airflow) to automatically generate updated forecasts and inventory recommendations weekly.
- **Interactive Dashboard:** Build a dashboard (using Streamlit or Plotly Dash) to allow managers to visualize forecasts, explore inventory levels, and simulate the impact of different cost assumptions.

7. Final Technical Stack Summary

- **Core Data Processing:** Polars (LazyFrame API)
- **Machine Learning Model:** LightGBM
- **Data Manipulation & Analysis:** Pandas, NumPy
- **Statistical Analysis:** SciPy
- **Visualization:** Plotly Express
- **Environment:** Kaggle Notebooks (Python 3)